How much SPICE can you get in 1000 lines of code?

nanospice: the classic Berkeley algorithm set in one Rust file

Milan Rother

2026

github.com/milanofthe/nanospice

Berkeley, 1969

- an integrated circuit cannot be breadboarded, and an internal node cannot be probed
- a design error after tapeout costs a mask set: simulation becomes the instrument
- Berkeley builds a series of simulators: BIAS, SLIC, TIME, CANCER

The machine

A CDC 6400 mainframe: 256 KB of memory by day, 384 KB at night. A large circuit is 50 nodes and 25 transistors. Jobs run in a nightly batch; a crash kills every job behind it.

CANCER

- a class project in Ron Rohrer's circuit simulation course; the grading rule: if Don Pederson approves of the program, everyone passes
- already inside: sparse matrix solver, implicit integration, built-in device models, adjoint sensitivity
- presented by Rohrer at the 1971 ISSCC

Computer Analysis of Nonlinear Circuits, Excluding Radiation

Simulators written under defense contracts were required to evaluate the radiation hardness of a circuit. The name was a statement that this program never would, and took no defense money. It was Berkeley in the sixties.

CANCER becomes SPICE

1971	Nagel renames it: Simulation Program with Integrated Circuit Emphasis
1972	released to the public domain, Pederson's condition
1973	announced at the Midwest Symposium on Circuit Theory, Waterloo
1975	SPICE2, Nagel's thesis: MNA, trapezoid with LTE, limiting, stepping
1983	SPICE 2G6 (Ellis Cohen): the netlist format freezes
1993	SPICE 3f5 (Tom Quarles): rewrite in C, gmin stepping
today	ngspice, Xyce; HSPICE, PSpice, Spectre, Eldo

- public domain is why SPICE won: universities taught it, graduates wrote to Berkeley for a tape, every derivative kept the language
- the MOSFET model arrived in a weekend when Dave Hodges came from Bell Labs: Shichman-Hodges, the level 1 of this talk

Line counts, measured

program	year	language	code lines
SPICE 2G6	1983	Fortran	14595
SPICE 3f5	1993	C	126697
ngspice	2026	C	562128
nanospice	2026	Rust	1000

- nonblank, noncomment lines, counted with tokei on the published sources; ngspice from the current git tree, C and headers only
- the algorithm core is a small fraction of any production simulator; the bulk is device models, UI, and compatibility

So: how much SPICE can you get in 1000 lines?

The rule

- one file, at most 1000 nonblank, noncomment lines, std only
- a test counts the lines and fails the build above the limit
- tests, comments, and example netlists are free

The budget forces every feature to displace another: the cost of each decision becomes explicit.

current count: **1000 of 1000**

What fits

Analyses

- `.op`, `.dc`, `.tran`, `.ac`

Devices

- R, C, L, V, I, E, G
- diode, MOSFET, JFET, BJT
- junction and gate capacitances

Solver

- sparse LU, partial pivoting
- Newton with junction limiting
- gmin and source stepping
- trapezoid, LTE, breakpoints

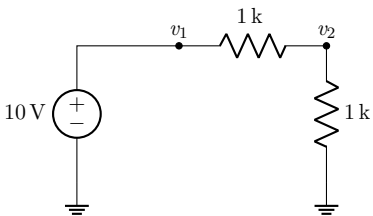
Input

- SIN, PULSE, PWL; `.model`, `.print`

Where the lines go

code section	lines	cumulative
constants and plumbing	23	23
numbers: `Num` trait, complex, suffixes	78	101
circuit data model	61	162
parser	275	437
sparse LU and stamp helpers	121	558
device models	64	622
solver	142	764
output selection	26	790
analyses and main	210	1000

Modified nodal analysis



$$\begin{pmatrix} g & -g & 1 \\ -g & 2g & 0 \\ 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \\ i \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 10 \end{pmatrix}$$

- one branch current per voltage-defined element (V, L, E)
- the branch row has a structural zero on the diagonal: pivoting is mandatory
- ground is a dummy row 0: stamps never special-case it

Sparse LU

`sparse LU and stamp helpers` 121 lines

- rows are sorted (column, value) vectors; elimination is a merge, fill-in falls out as the union

The first-entry invariant

Every active row contains only columns $\geq k$: a column-k entry is always the *first* element, so rows bucket by their first column and pivot search reads one bucket. The factorization is linear on ladders and trees.

- a singular matrix names its column: `singular matrix at node b, no conduction path?`

Two primitives carry every device

```
fn contrib(&mut self, p: usize, n: usize, i: T,
           ctl: &[(usize, usize, T)], x: &[T]) {
    let mut ieq = i;
    for &(a, b, g) in ctl {
        self.quad(p, n, a, b, g);
        ieq = ieq - g * (x[a] - x[b]);
    }
    self.rhs(p, T::zero() - ieq);
    self.rhs(n, ieq);
}
```

The contribution operator of Verilog-A, $I(p, n) <+ f(v)$; the dual `vcontrib` writes the branch row of the voltage-defined elements.

The whole device set

device	primitive	controls
R, C, I	`contrib`	self conductance
G (VCCS)	`contrib`	one control pair
diode	`contrib`	one junction
MOSFET, JFET	`contrib`	g_m, g_{ds}
BJT	3 × `contrib`	two junctions, transport
V, L, E	`vcontrib`	none, own current, control pair

- nmos/pmos and source-drain swap in one evaluation: every derivative picks up the polarity twice, $s^2 = 1$, the stamps are sign-free
- one Jacobian structure in one place; hand-derived stamps become verification artifacts

Junction limiting

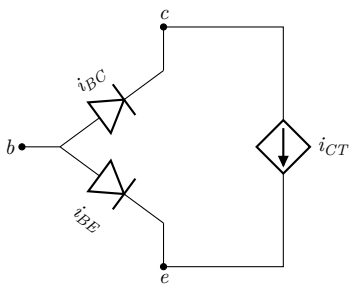
$$v_{crit} = V_T \ln \frac{V_T}{\sqrt{2}\, I_S} \qquad\qquad v \leftarrow v_{old} + V_T \ln\!\left(1 + \frac{v_{new} - v_{old}}{V_T}\right)$$

- Newton diverges on the bare exponential: 5 V proposed means e^{193} in the Jacobian
- beyond the curvature maximum v_{crit}, steps become logarithmic
- this is SPICE2's `pnjlim`, unchanged since 1975

Failure mode: limiting without the flag

A common-emitter amplifier converges with the transistor off: the limiter clamps the junction, no current flows, the deltas vanish, the test says done. The rule: an iteration in which a limiter changed a voltage cannot converge. A delta test alone cannot see active limiting.

BJT: three contributions




```
s.contrib(*b, *e, .., &[(*b, *e, gpi)], x);
s.contrib(*b, *c, .., &[(*b, *c, gmu)], x);
s.contrib(*c, *e, ..,
  &[(*b, *e, gmf), (*b, *c, -gmr)], x);
```

KCL composes the classic 3×3 stamp; rows and columns sum to zero by construction.

Junction capacitances by desugaring

$$C(v) = C_{j0}\left(1 - \tfrac{v}{V_J}\right)^{-m}, \qquad Q(v) = \frac{C_{j0}\,V_J}{1-m}\left(1 - \left(1 - \tfrac{v}{V_J}\right)^{1-m}\right)$$

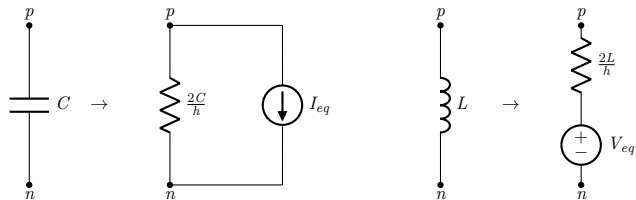
- `cjo`, `cje`/`cjc`, `cgs`/`cgd` desugar into internal capacitors in the parser
- no assembly, state, timestep, or AC code knows they exist
- charge-based, graded ($m = 0.5$) for junctions, linear for gate oxide

Operating point fallbacks are data

```
let gmin_path: Vec<(f64, f64)> =
    (2..=12).map(|k| (10f64.powi(-k), 1.0)).collect();
let src_path: Vec<(f64, f64)> =
    (1..=10).map(|k| (GMIN, k as f64 / 10.0)).collect();
for path in [gmin_path, src_path] {
    if path.iter().all(|&(g, fac)|
        newton(ckt, x, lim, &Mode::Dc { fac }, g, ITL_OP).is_some()) {
        return;
    }
}
```

Both classic fallbacks are homotopy paths in the (g_{min}, source scale) plane, walked with warm starts.

Transient: companion models



- state per reactive element: charge or flux plus the dual rate, one update rule for both
- trapezoid: second order, A-stable, undamped

Adaptive timestep

$$\varepsilon_{tr} \approx \frac{h^3/12}{h(h+d_1)(h+d_1+d_2)/6 + h^3/12}\,\big(x_c - x_p\big), \qquad h' = 0.9\,h\,r^{-1/3}$$

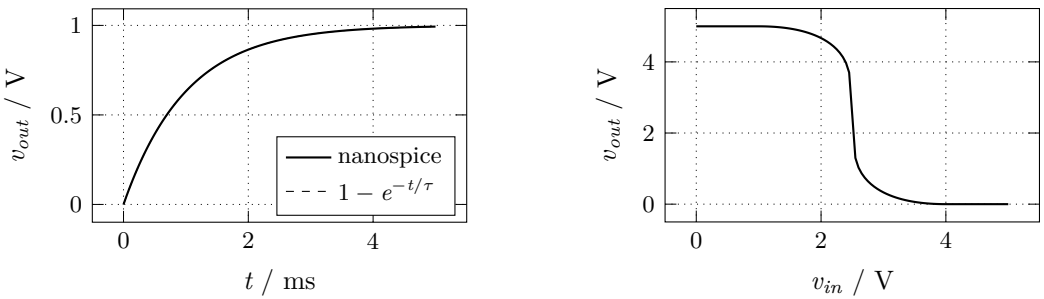
- quadratic predictor through the last three points; corrector minus predictor measures $\dddot{x}$ (uniform steps: the classic 1/13)
- the predictor doubles as the Newton starting point
- waveform corners are breakpoints; steps land on them exactly
- the two predictor-less restart steps run damped backward Euler
- when the cuts bottom out: `timestep too small`; SPICE2 said `TIMESTEP REDUCED TO ZILCH`

Small-signal AC

$$(G + j\omega C)\,\hat{x} = \hat{b}$$

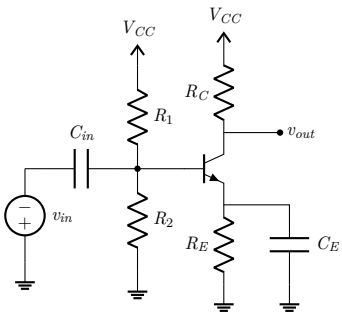
- at a converged operating point the limiters pass voltages through: *G is the DC Jacobian the last Newton iteration assembled*
- the AC path reuses `assemble`, maps to complex, adds $j\omega C$ and $-j\omega L$
- one generic sparse LU over a three-method `Num` trait serves Newton and AC; no device model contains AC-specific code

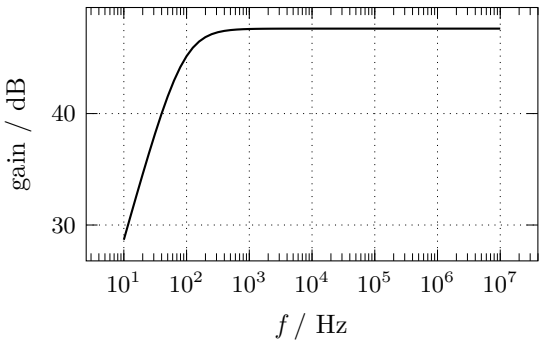
Results: step response and inverter



RC transient against the closed form; CMOS inverter `.dc` transfer

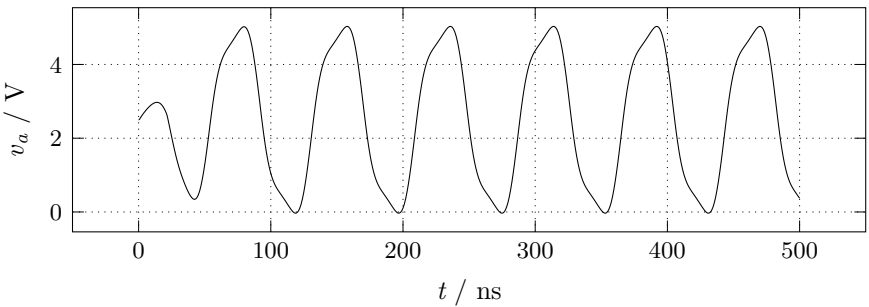
Results: common-emitter amplifier





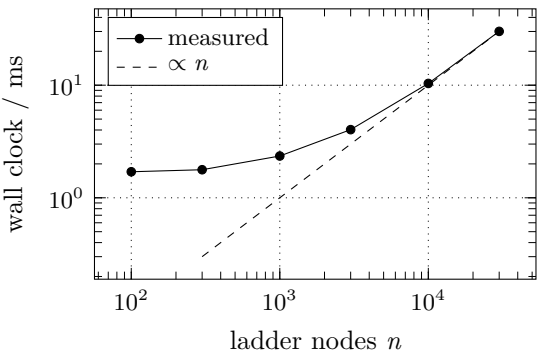
midband $g_m R_C \approx 240$, exactly 47.6 dB in the flat region

Results: it oscillates



- three CMOS inverters, gate capacitances set the stage delay
- period 78 ns, stable within 1.5 %; a test bounds the jitter
- without the capacitances the ring has no defined frequency

Benchmark: ladder operating points



- below 10^3 nodes the 1.7 ms process startup dominates
- linear through the last point: 30 ms for 30000 nodes
- the bucketed pivot search keeps the whole factorization at the fill-in bound; a row scan would be $O(n^2)$

Validation

`tests/cli.rs` 330 lines, 0 of 1000: tests are free

23 integration tests, references independent of the implementation:

- analytic: step responses, AC corners, LC amplitude and energy
- bias points: MOSFET, JFET, BJT, exact npn/pnp symmetry
- structural: randomized ladder vs. a Thevenin reduction computed in the test
- negative: malformed decks produce named errors, never a panic

The cut list

feature	why it lost	est. lines
`.subckt` hierarchy	lost to the transistor models	∼100
Markowitz ordering	netlist order suffices	∼80
Gummel-Poon BJT	Ebers-Moll covers the core	∼80
noise analysis	needs the adjoint system	∼80
`.param` expressions	a calculator is its own project	∼60

The most consequential remaining cut is `.subckt`: real decks are hierarchical.

How much SPICE in 1000 lines? The 1975 core, complete.

Sparse LU, two contribution primitives, Newton with limiting and homotopy fallbacks, trapezoid with LTE, breakpoints and damped restarts, graded junction capacitances, AC from the DC Jacobian, eleven devices, four analyses, 23 tests. The algorithm set is fifty years old and still carries a working simulator.

`github.com/milanofthe/nanospice`